Mount Kenya University

Unlocking Infinite Possibilities

NAME : STEPHEN KIMANI

ADMMISSION: BSCCS/2024/57444

COURSE : BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNIT CODE: BIT 4138

UNIT NAME: ADVANCED CRYPTOGRAPHY

LECTURER: MICHAEL NYORO

SEMESTER /ACADEMIC YEAR: SEMESTER 6

PROJECT DETAILS: IMPLEMENATION AND ANALYSIS OF CRYPTOGRAPHIC ALGORITHMS USING PYTHON

Technologies Used:

I. **Python 3**
II. **Visual Studio Code**
III. **Git**
IV. **OpenSSL**
V. **PyCryptodome**
VI. **Windows PowerShell / Command Prompt**

ONLINE PORTFOLIO/ GITHUB LINK

https://github.com/Kimani985/BIT4138-Advanced-Cryptography

Contents

WEEK 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

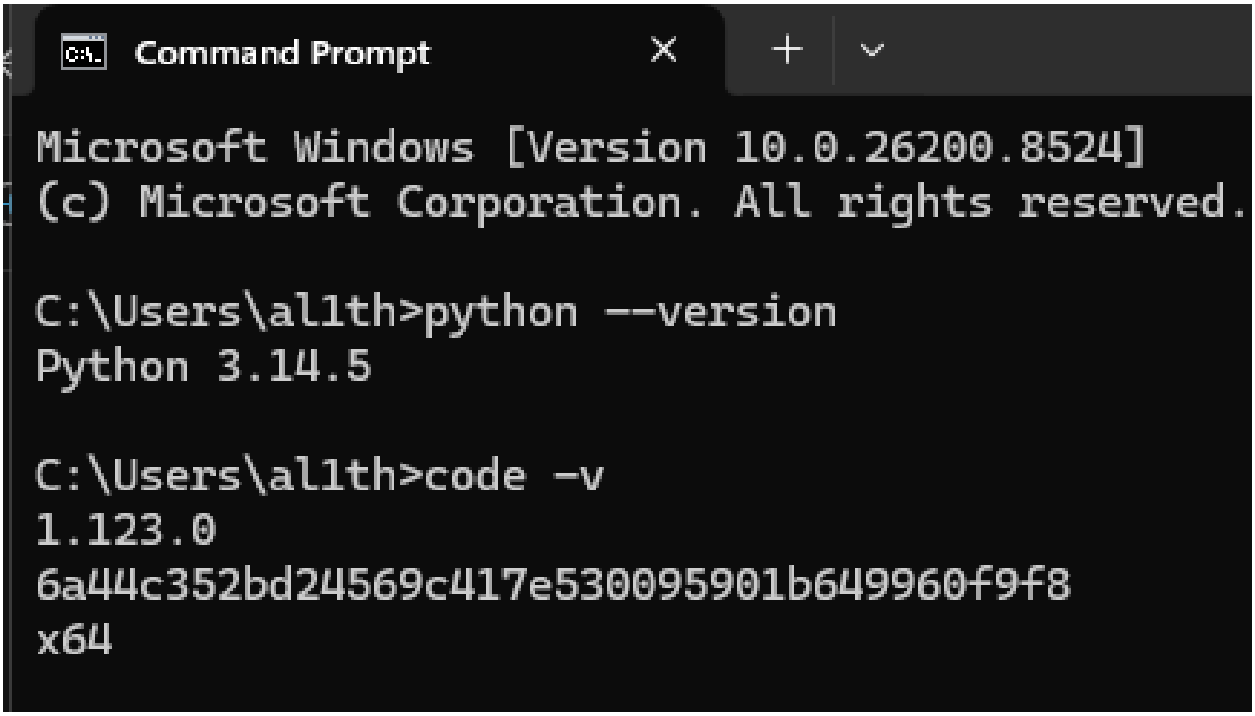


Figure 1: Successful installation of Python and Visual Studio Code for developing and testing cryptographic applications.

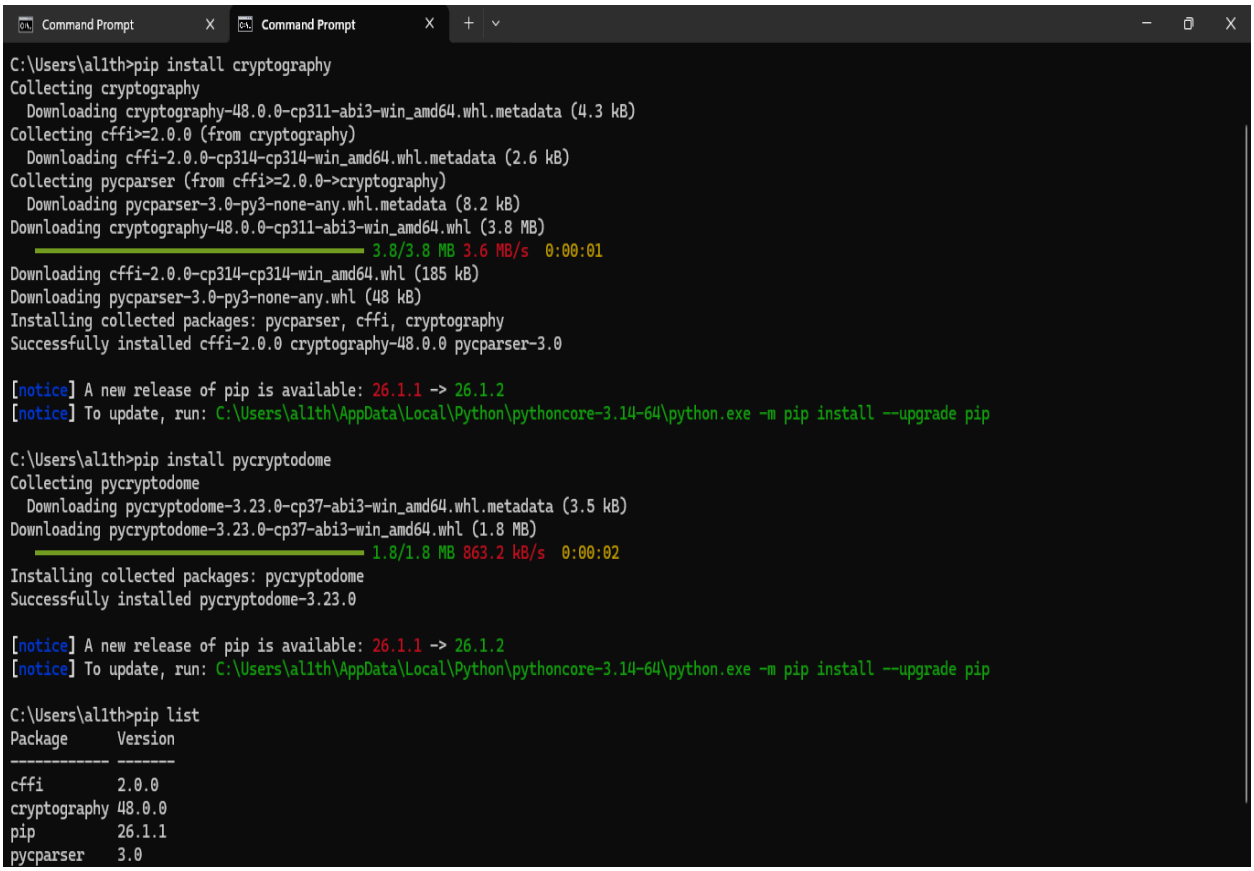


Figure 2: Installation of cryptography libraries using pip, enabling implementation of encryption and decryption algorithms.

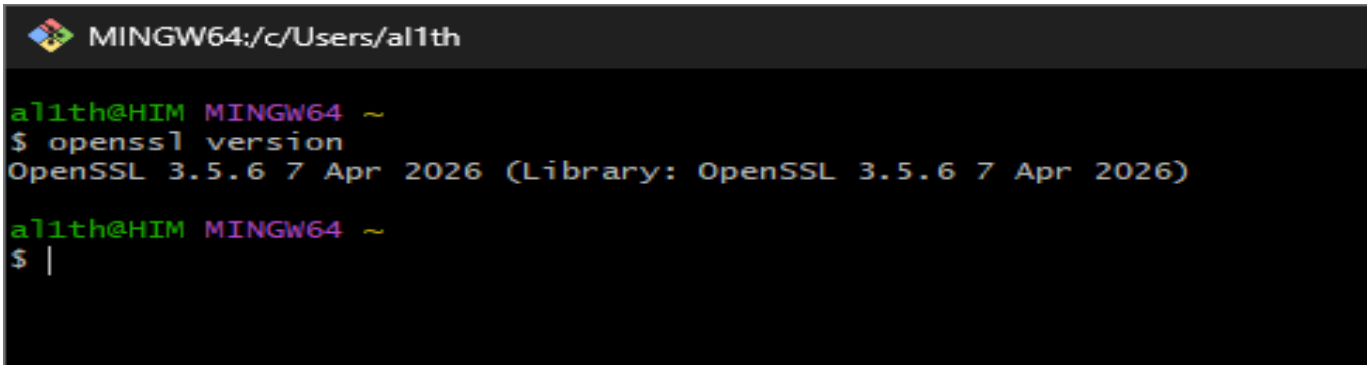


Figure 3: OpenSSL version verification confirming successful configuration of cryptographic command-line tools.

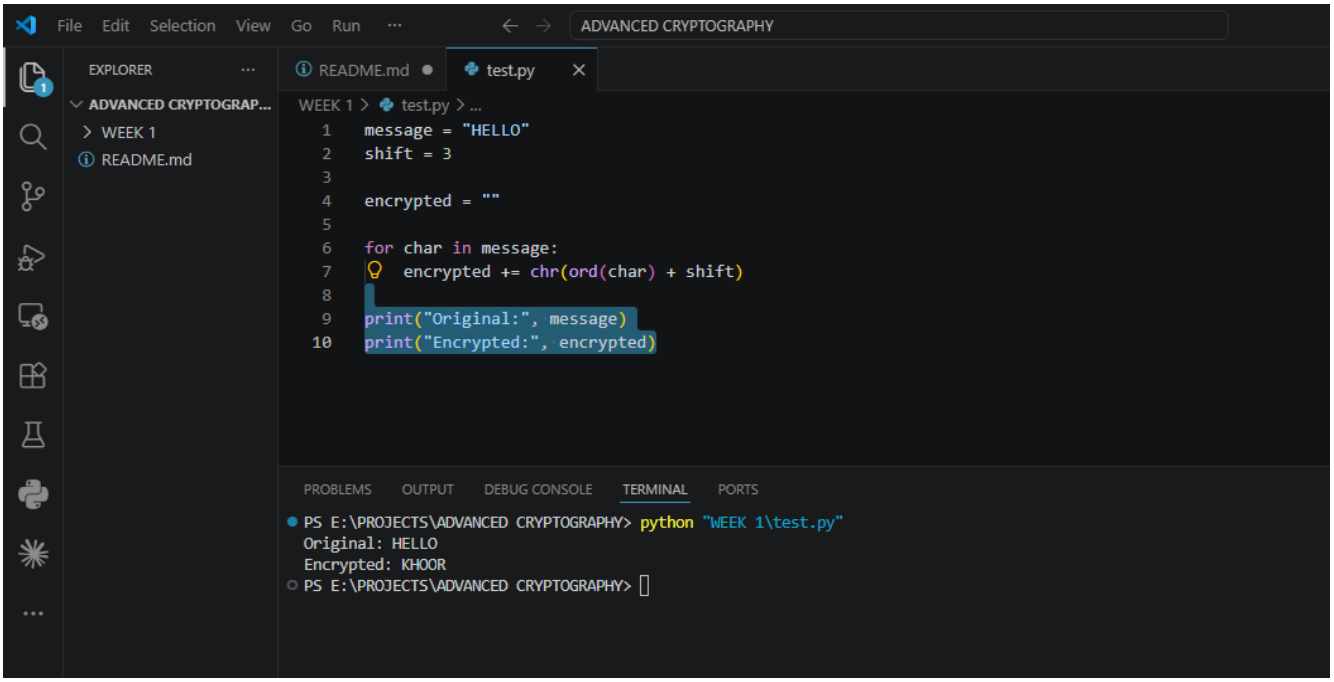


Figure 4: Execution of a Python test script verifying the development environment is operational.

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git init
Initialized empty Git repository in E:/PROJECTS/ADVANCED CRYPTOGRAPHY/.git/
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git add .
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git commit -m "Week 1 environment setup completed"
[main (root-commit) 2652837] Week 1 environment setup completed
 2 files changed, 10 insertions(+)
 create mode 100644 README.md
 create mode 100644 WEEK 1/test.py
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 5: Initialization of GitHub repository for version control and project documentation.

CREATIVE TASK - **Environment Checker**

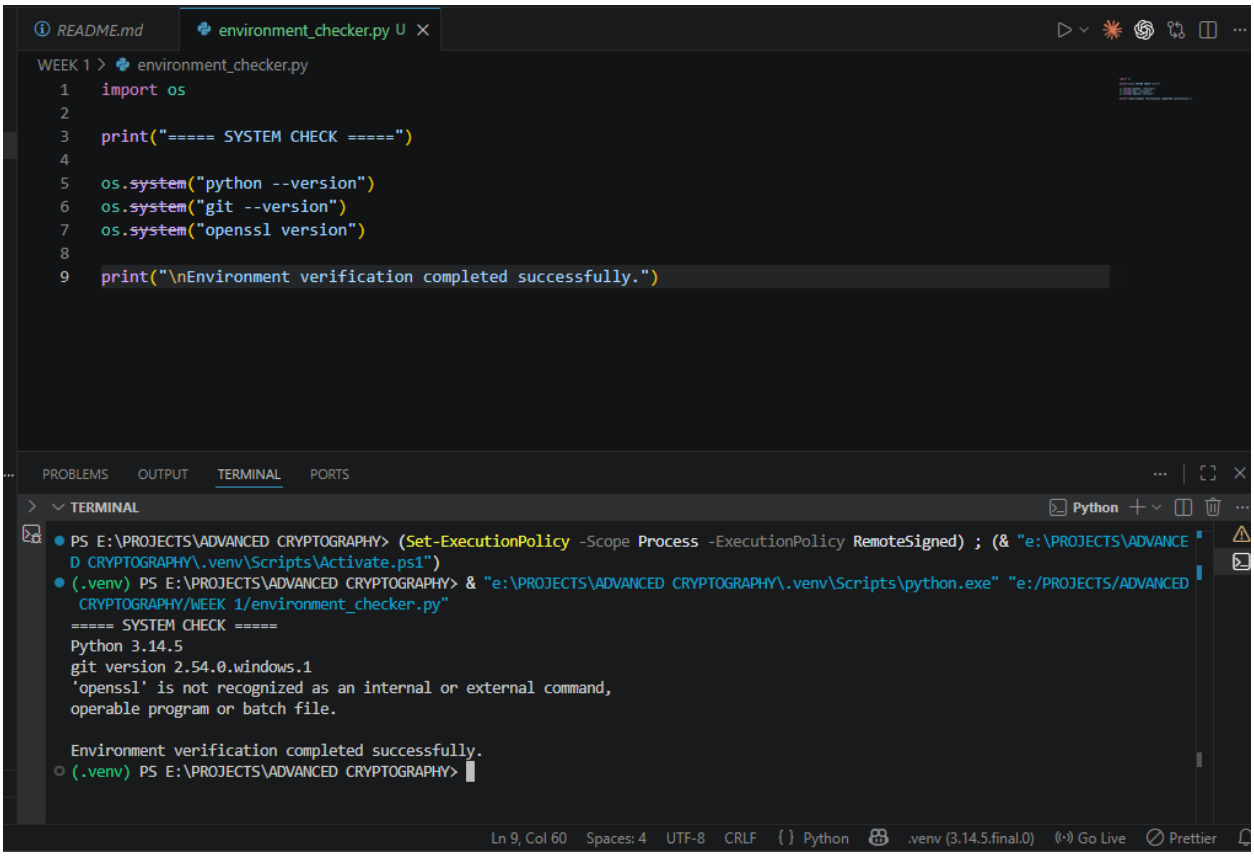


Figure 6: Automated environment verification tool displaying the installed versions of Python, Git, and OpenSSL to confirm system readiness for cryptographic development.

Reflection

This week focused on setting up the cryptography development environment. Python, VS Code, OpenSSL, and required libraries were successfully installed and tested. GitHub was configured for version control. The environment is now ready for implementing and testing cryptographic algorithms.

WEEK 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

WEEK 2 > classical_cipher.py > ...

```python
def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isalpha():
            result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
        else:
            result += char

    return result


def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)


message = input("Enter message: ")

if not message.replace(" ", "").isalpha():
    print("Invalid input! Use letters only.")
else:
    shift = int(input("Enter shift value: "))

    encrypted = caesar_encrypt(message, shift)
    decrypted = caesar_decrypt(encrypted, shift)

    print("\nOriginal:", message)
    print("Encrypted:", encrypted)
    print("Decrypted:", decrypted)
```

Figure 7: Caesar Cipher implementation demonstrating character shifting using a fixed encryption key.

Figure 8: Successful encryption and decryption outputs generated by the implemented cipher algorithm.

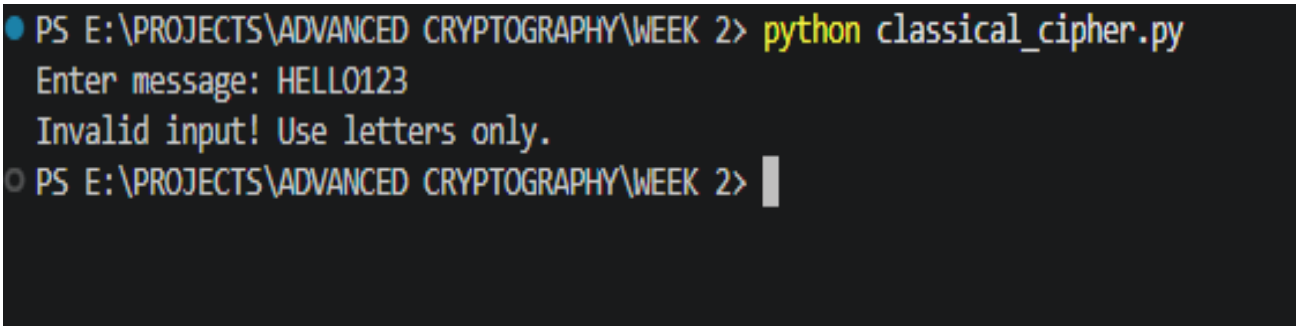
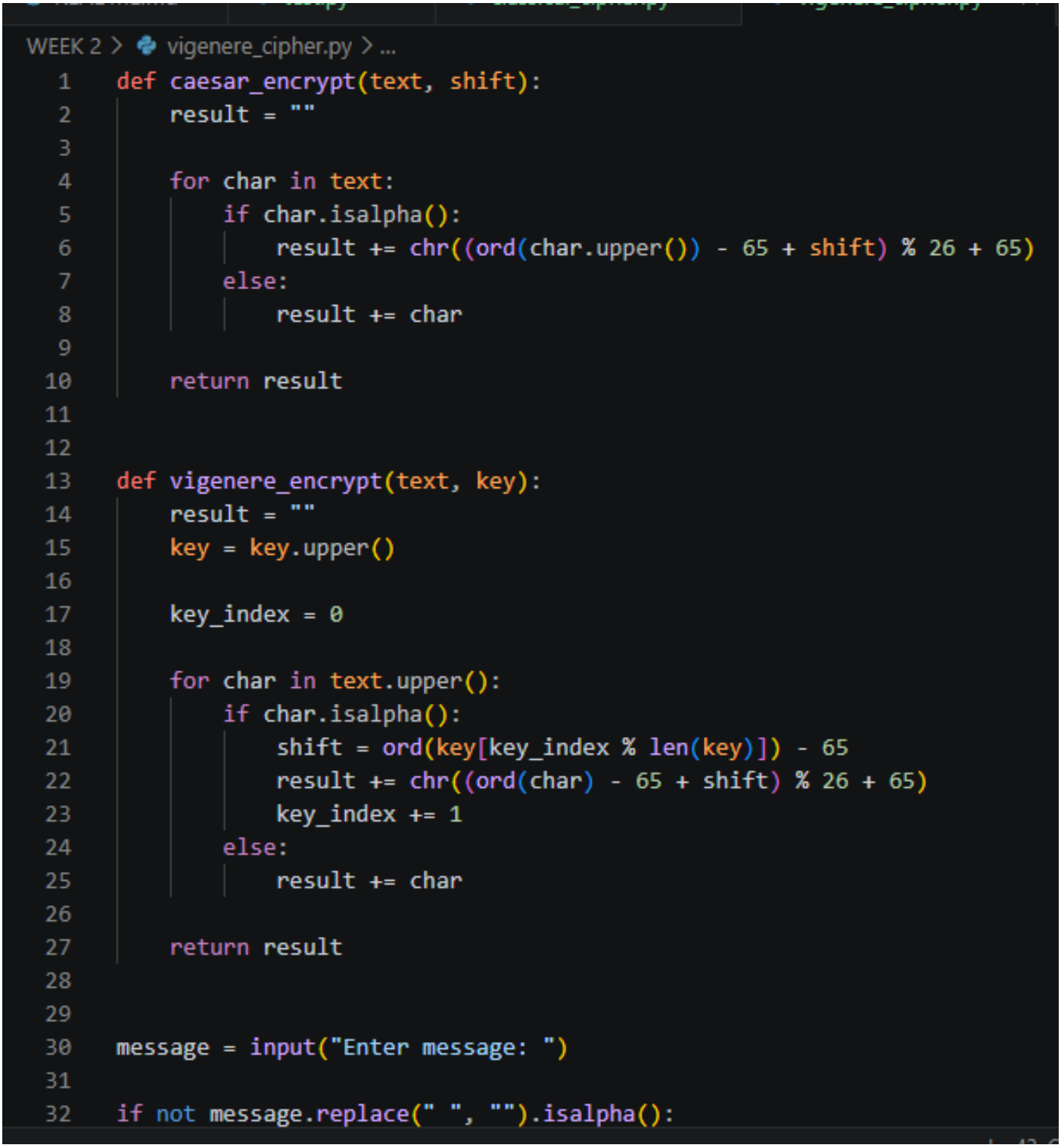


Figure 9: Input validation process preventing invalid plaintext entries during encryption.



WEEK 2 > vigenere_cipher.py > ...

```
1   def caesar_encrypt(text, shift):
2       result = ""
3
4       for char in text:
5           if char.isalpha():
6               result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
7           else:
8               result += char
9
10      return result
11
12
13  def vigenere_encrypt(text, key):
14      result = ""
15      key = key.upper()
16
17      key_index = 0
18
19      for char in text.upper():
20          if char.isalpha():
21              shift = ord(key[key_index % len(key)]) - 65
22              result += chr((ord(char) - 65 + shift) % 26 + 65)
23              key_index += 1
24          else:
25              result += char
26
27      return result
28
29
30  message = input("Enter message: ")
31
32  if not message.replace(" ", "").isalpha():
```

Figure 10: Vigenère Cipher implementation demonstrating keyword-based encryption.

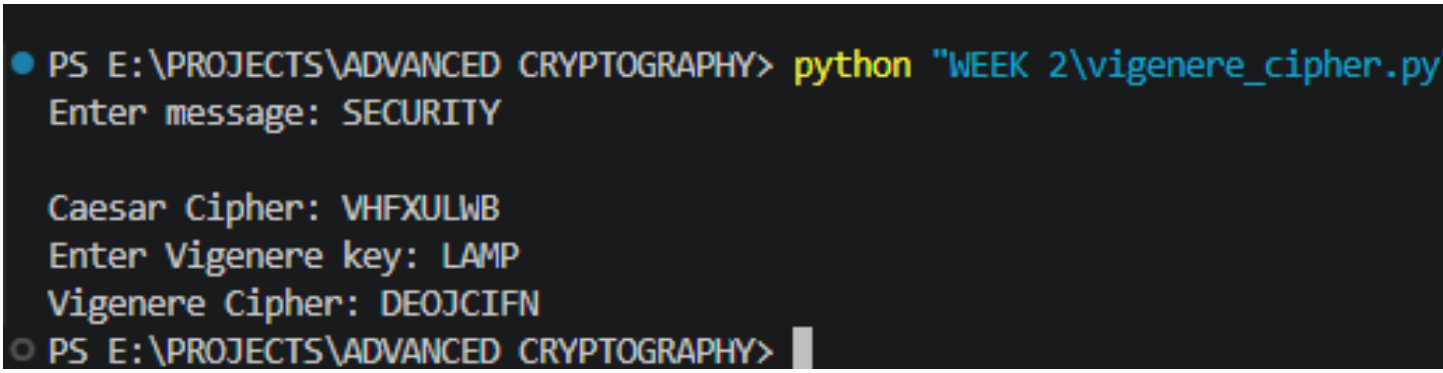


Figure 11: Cipher testing results obtained using different messages and encryption keys.

CREATIVE TASK - Cryptography Toolkit

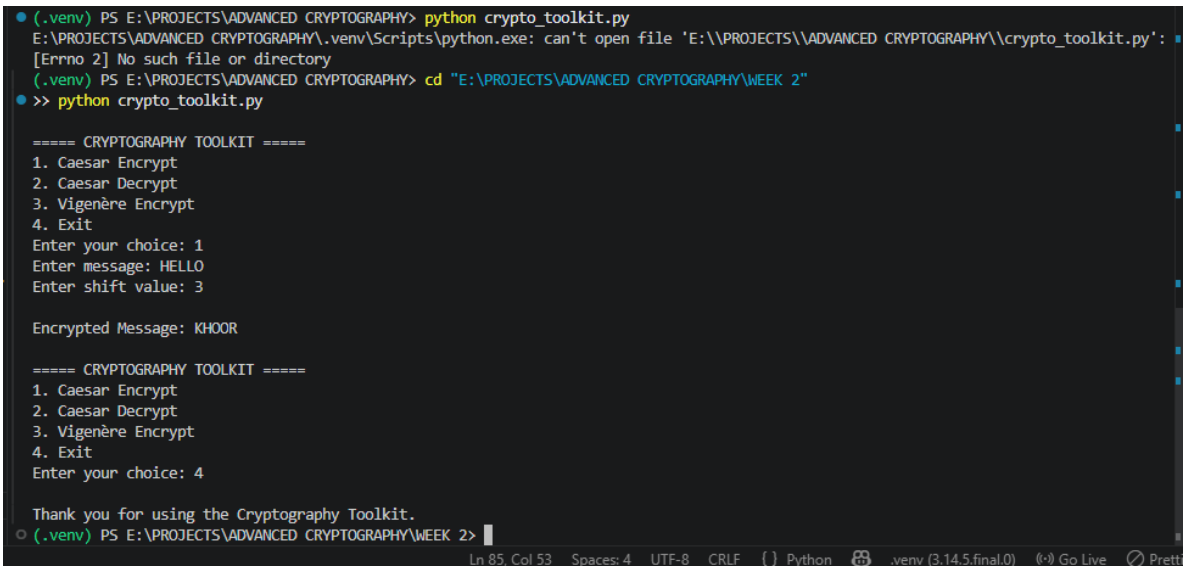


Figure 12: Interactive cryptography toolkit providing a menu-driven interface for selecting and executing classical encryption algorithms.

Reflection

The implementation of Caesar and Vigenère ciphers demonstrated the importance of encryption keys in protecting information. Testing showed that Vigenère Cipher provides stronger security through keyword-based encryption, while Caesar Cipher remains vulnerable to simple cryptanalysis techniques.

WEEK 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

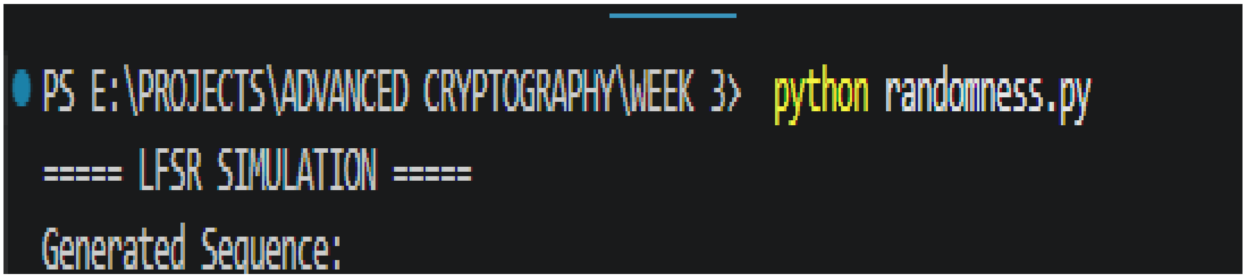


Figure 13: Implementation of a Linear Feedback Shift Register (LFSR) used to generate pseudorandom sequences.

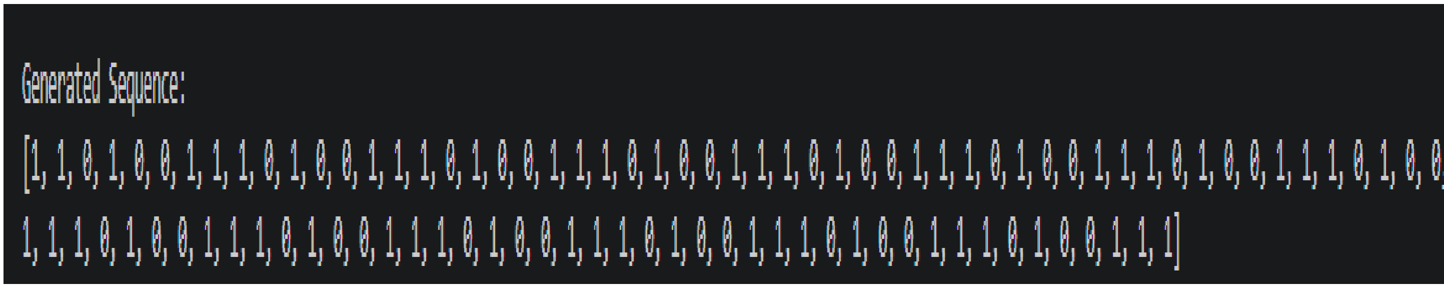


Figure 14: Generated pseudorandom binary sequence produced by the implemented LFSR.

```
===== RANDOMNESS TEST =====
Number of Ones  : 58
Number of Zeros: 42
Percentage of Ones  : 58.00%
Percentage of Zeros: 42.00%
Result: Weak Randomness
```

Figure 15: Statistical randomness testing results showing distribution of generated binary values.

```
===== RC4 STYLE SIMULATION =====
Message: HELLO
Key Stream: [153, 253, 3, 156, 58]
Encrypted: [209, 184, 79, 208, 117]
```

Figure 16: RC4-style stream cipher simulation demonstrating encryption using a generated keystream.

```
===== PERFORMANCE RESULTS =====
Execution Time: 7.7247619628906 25e-05 seconds
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3>
```

Figure 17: Performance evaluation results showing execution time during encryption operations.

CREATIVE TASK- Randomness Challenge Game

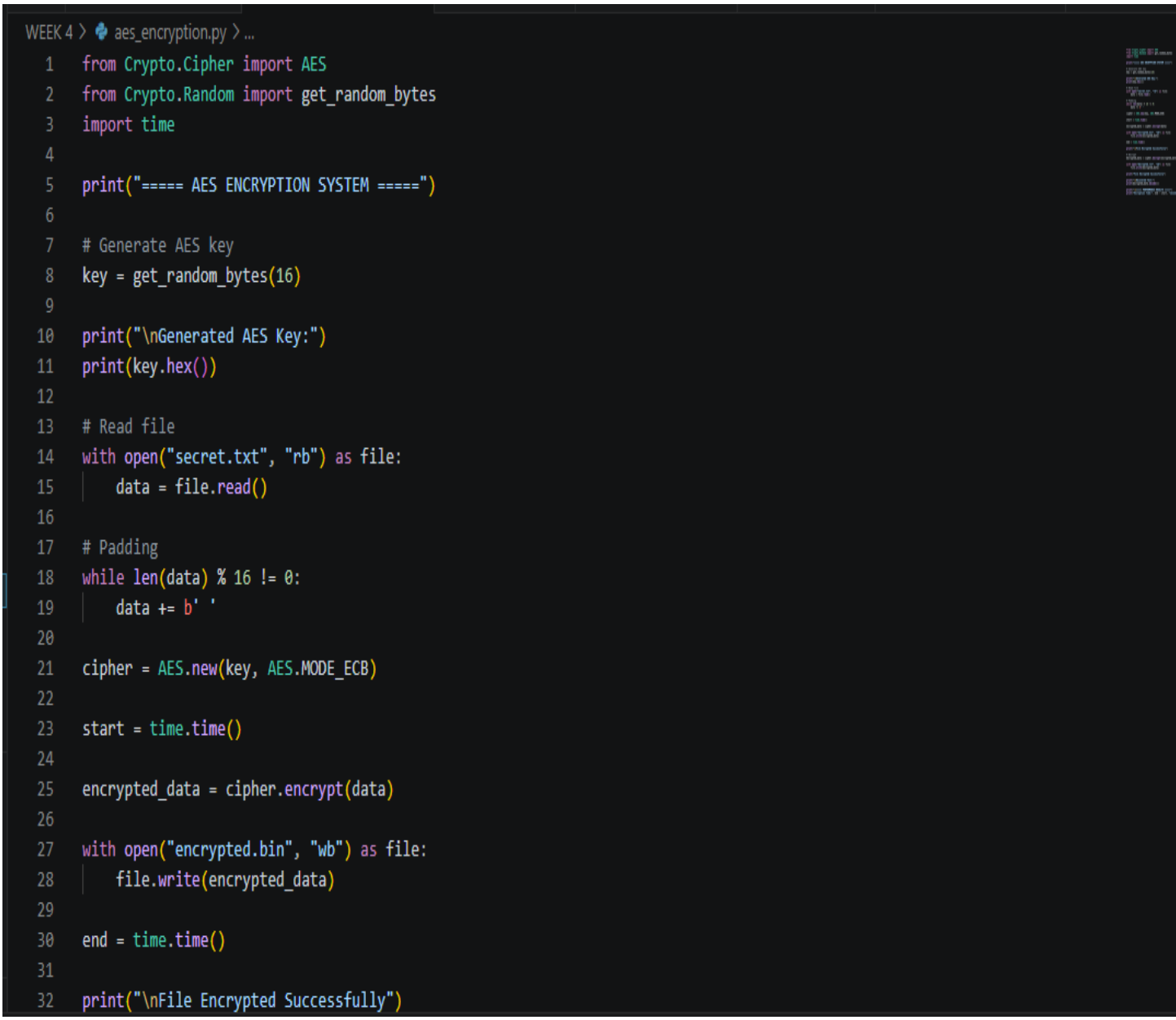
Figure 18: Interactive Randomness Challenge Game demonstrating binary sequence generation, user prediction, and statistical analysis of pseudorandom output.

Reflection

This week focused on pseudorandom sequence generation and stream cipher concepts. An LFSR was implemented to generate binary sequences, and statistical testing was performed to evaluate randomness. An RC4-style simulation demonstrated how stream ciphers use keystreams to encrypt data efficiently.

WEEK 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations



```
WEEK 4 > aes_encryption.py > ...
1  from Crypto.Cipher import AES
2  from Crypto.Random import get_random_bytes
3  import time
4
5  print("===== AES ENCRYPTION SYSTEM =====")
6
7  # Generate AES key
8  key = get_random_bytes(16)
9
10 print("\nGenerated AES Key:")
11 print(key.hex())
12
13 # Read file
14 with open("secret.txt", "rb") as file:
15     data = file.read()
16
17 # Padding
18 while len(data) % 16 != 0:
19     data += b' '
20
21 cipher = AES.new(key, AES.MODE_ECB)
22
23 start = time.time()
24
25 encrypted_data = cipher.encrypt(data)
26
27 with open("encrypted.bin", "wb") as file:
28     file.write(encrypted_data)
29
30 end = time.time()
31
32 print("\nFile Encrypted Successfully")
```

Figure 19: AES encryption program implementing symmetric key cryptography for secure file protection

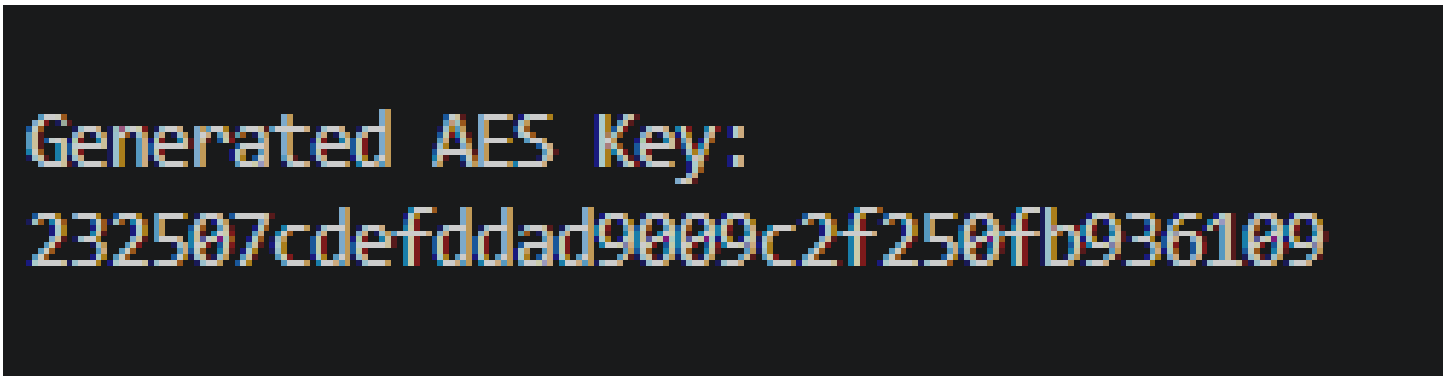


Figure 20: AES symmetric key generation process used for encryption and decryption operations

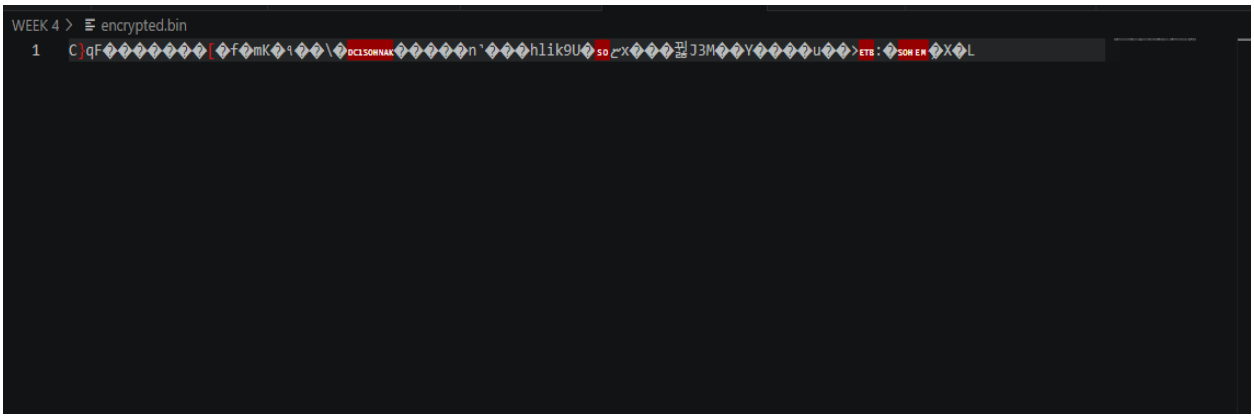


Figure 21: Successful encryption of a text file using the generated AES key.

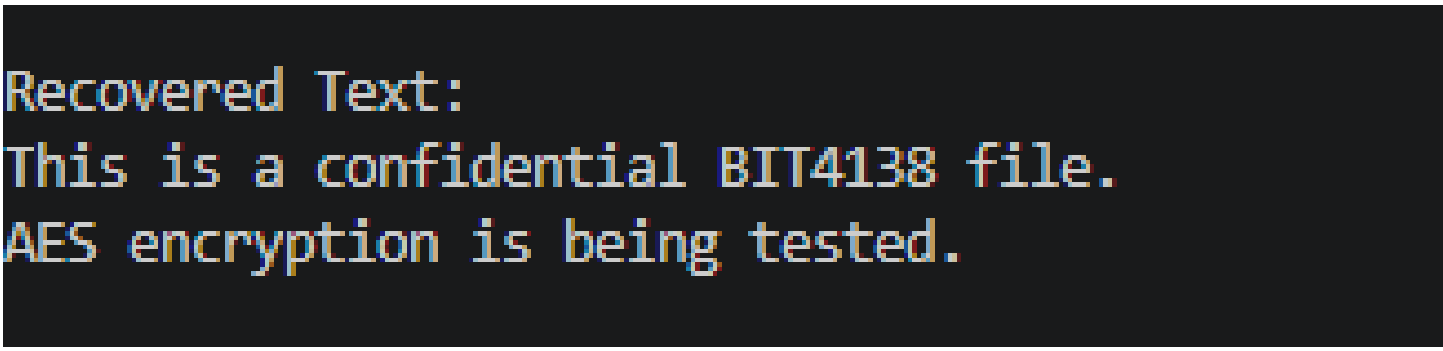


Figure 22: Successful decryption process restoring the original file contents.

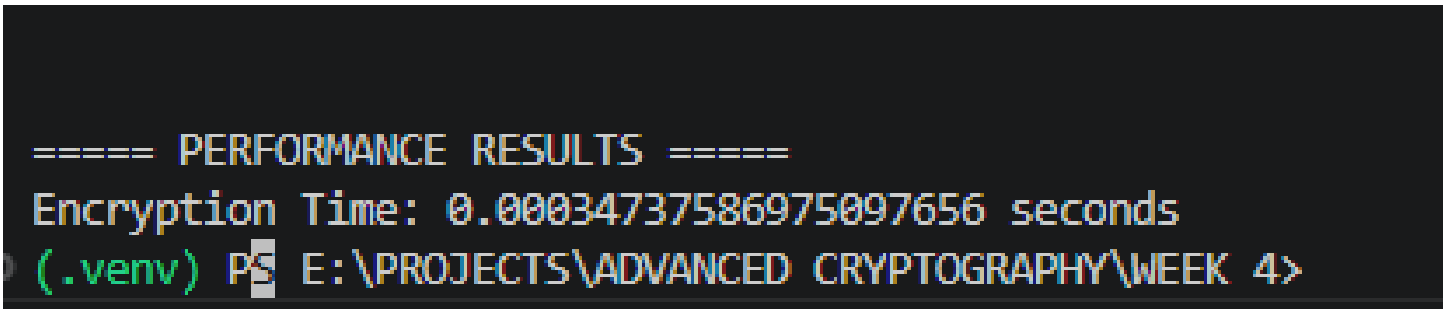


Figure 23: AES performance evaluation showing execution time during encryption.

CREATIVE TASK - Secure File Locker

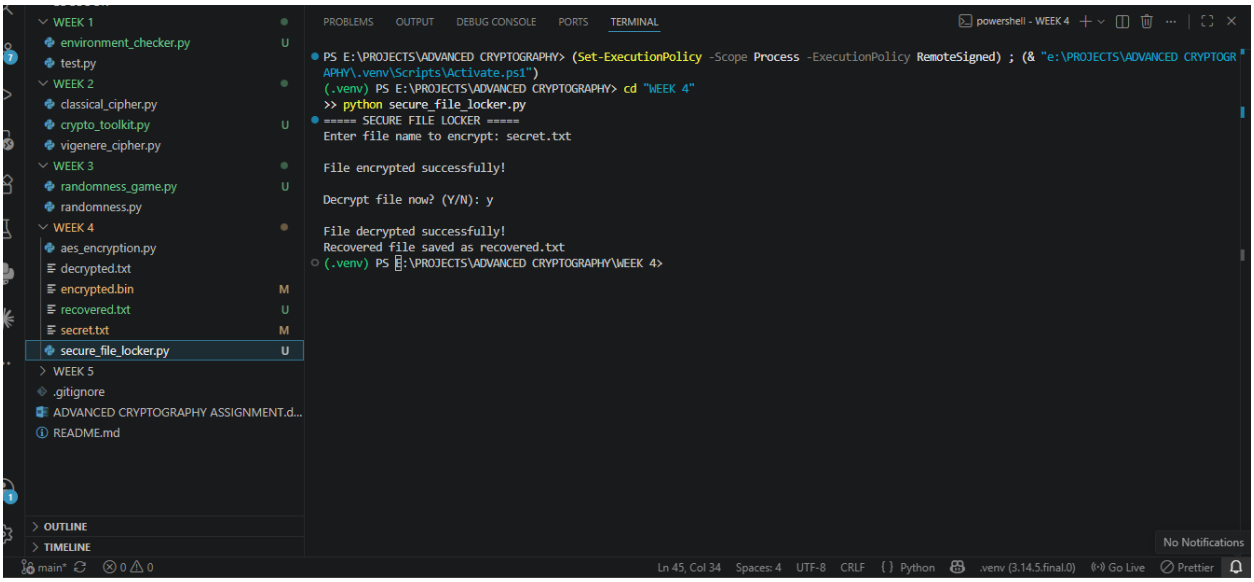


Figure 24: Secure File Locker demonstrating user-driven AES encryption and successful recovery of encrypted file contents through decryption.

Reflection

AES encryption was implemented to protect files using symmetric cryptography. Secure key generation, encryption, and decryption processes were successfully tested. The practical activity demonstrated the importance of key management and the effectiveness of AES in securing sensitive information.

WEEK 5: Public Key Cryptography (RSA)

Implementation of RSA encryption and Key Management

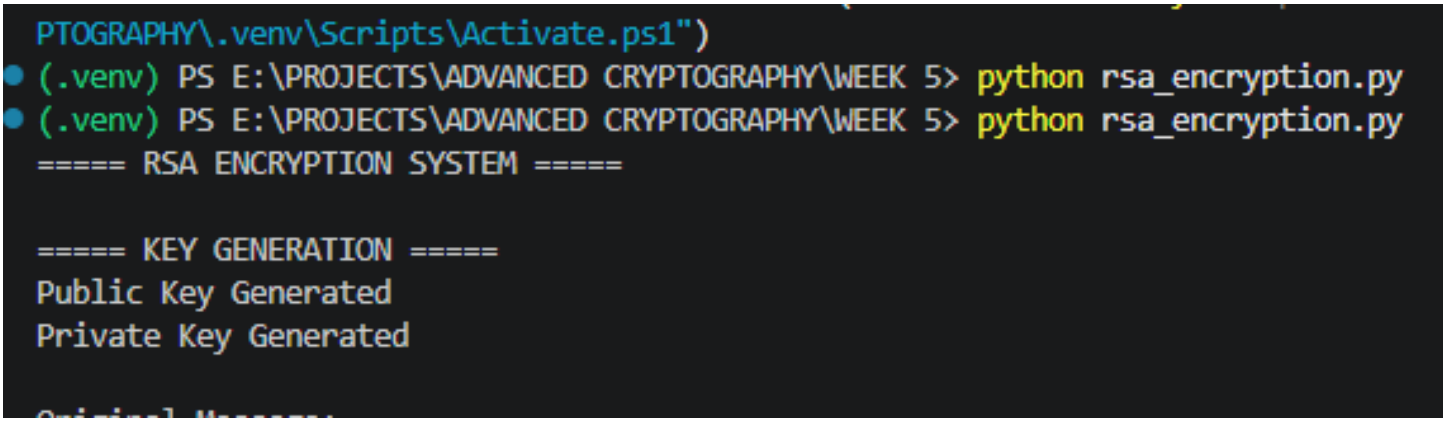


Figure 25: Generation of RSA public and private key pairs used for secure communication.

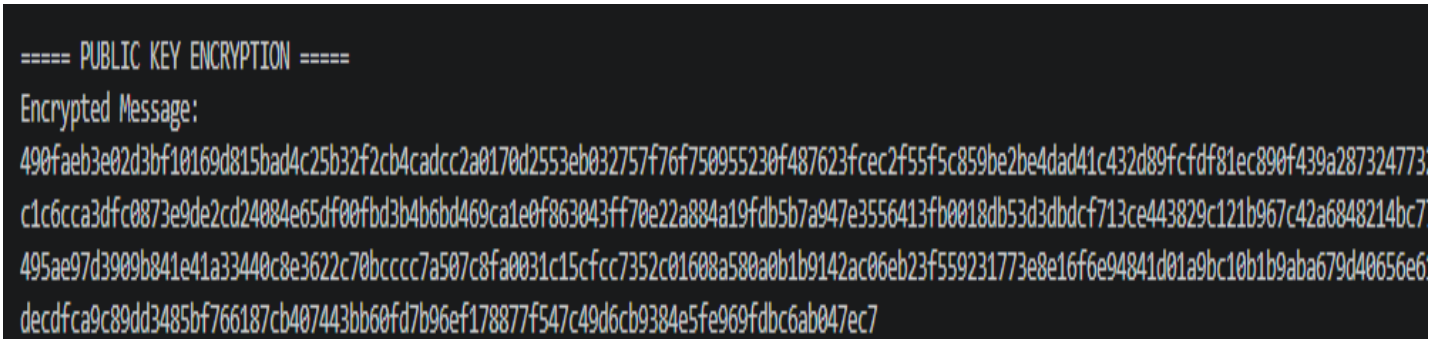


Figure 26: Encryption of plaintext using the RSA public key.

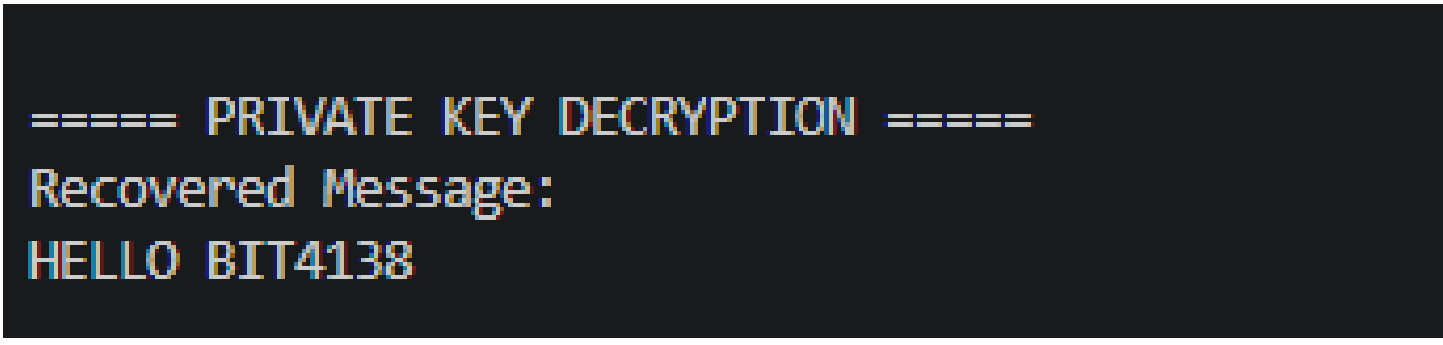


Figure 27: Successful recovery of the original message using the RSA private key.

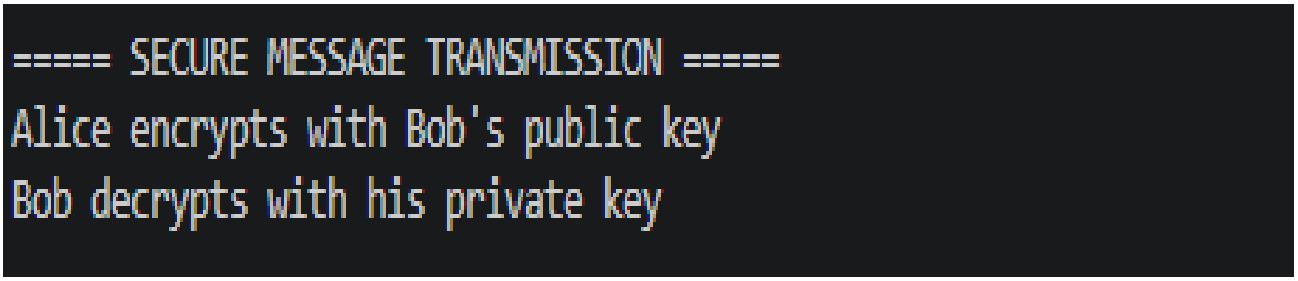


Figure 28: Simulation of secure communication using RSA public and private key cryptography.

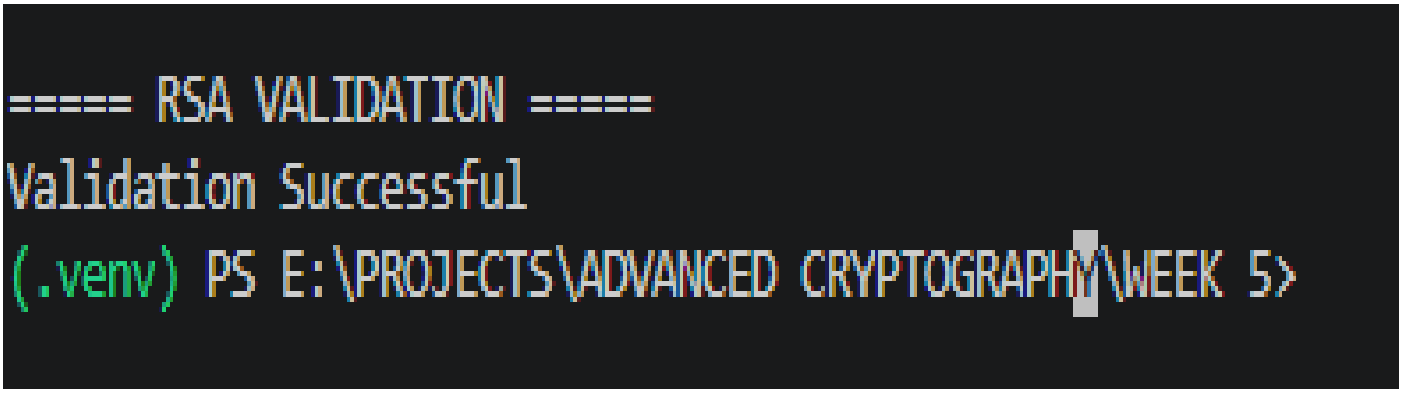


Figure 29: RSA testing and validation confirming successful encryption and decryption operations.

CREATIVE TASK - Secure RSA Messenger

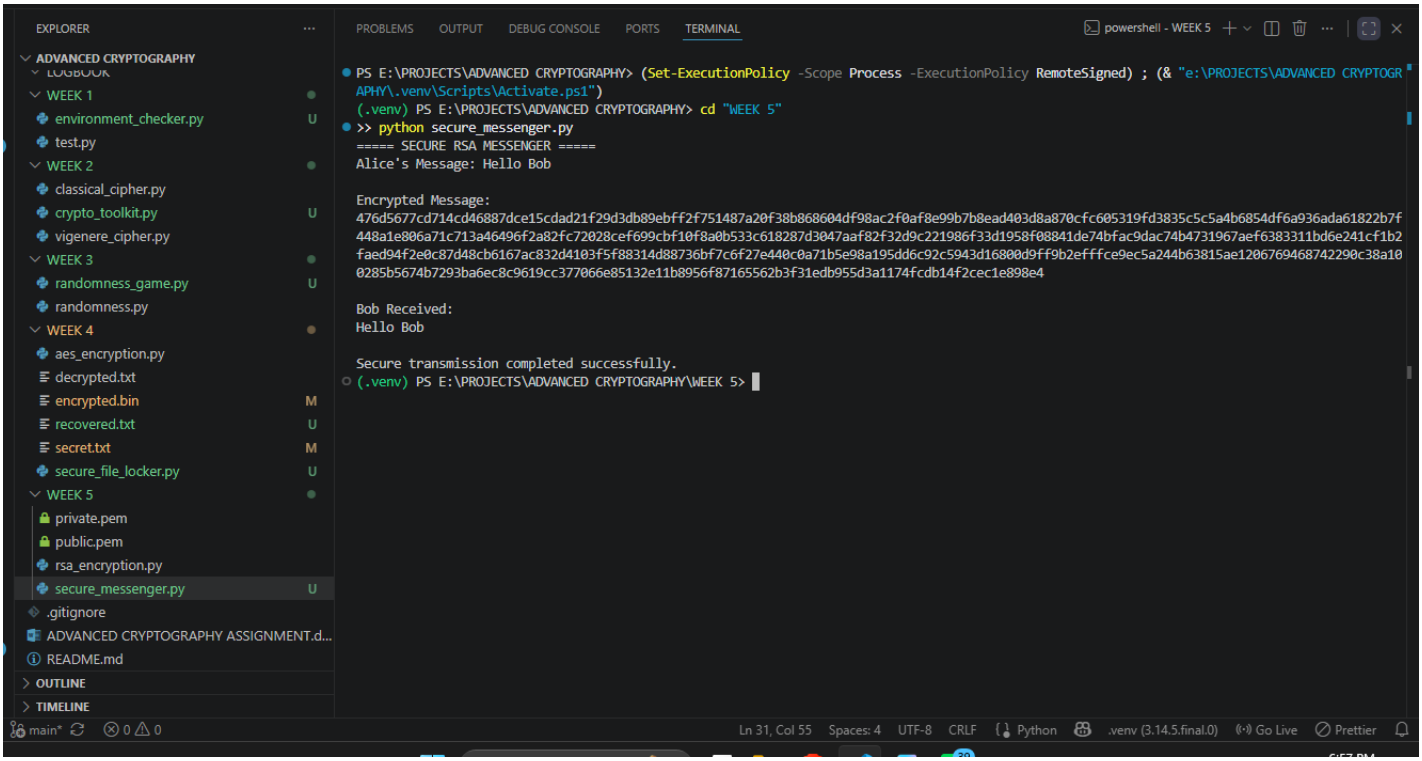


Figure 30: Secure RSA Messenger demonstrating encrypted communication between Alice and Bob through RSA public-key encryption and private-key decryption.

Reflection

RSA encryption demonstrated the use of public and private keys for secure communication. The Secure RSA Messenger successfully encrypted and decrypted messages, highlighting the practical application of asymmetric cryptography.

WEEK 6 - SPN, S-Boxes and Non-Linearity

Class Activity 1: Manual SPN Simulation

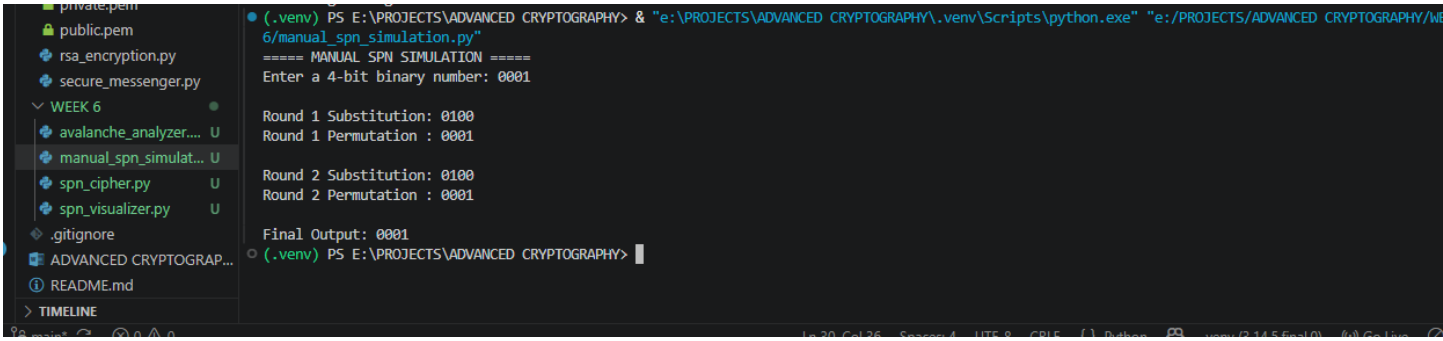


Figure 31: Manual SPN simulation showing substitution and permutation across two rounds.

Class Activity 2 – S-Box Analysis

Figure 32 : S-Box analysis showing mapping uniqueness and security evaluation

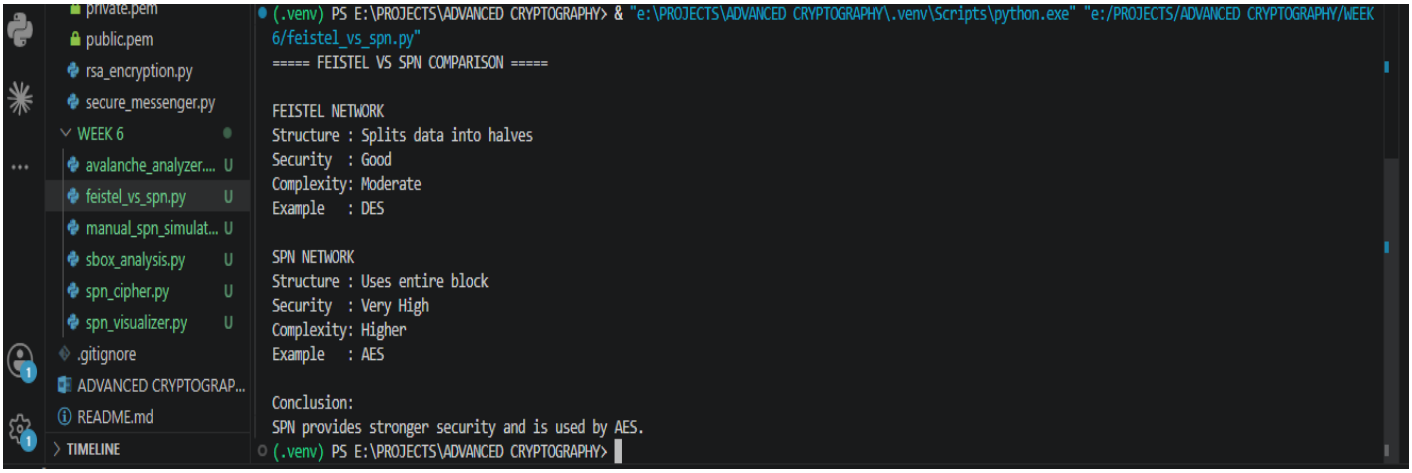


Figure 33: Feistel and SPN structure comparison.

Implementation of a Simple Substitution-Permutation Network (SPN)

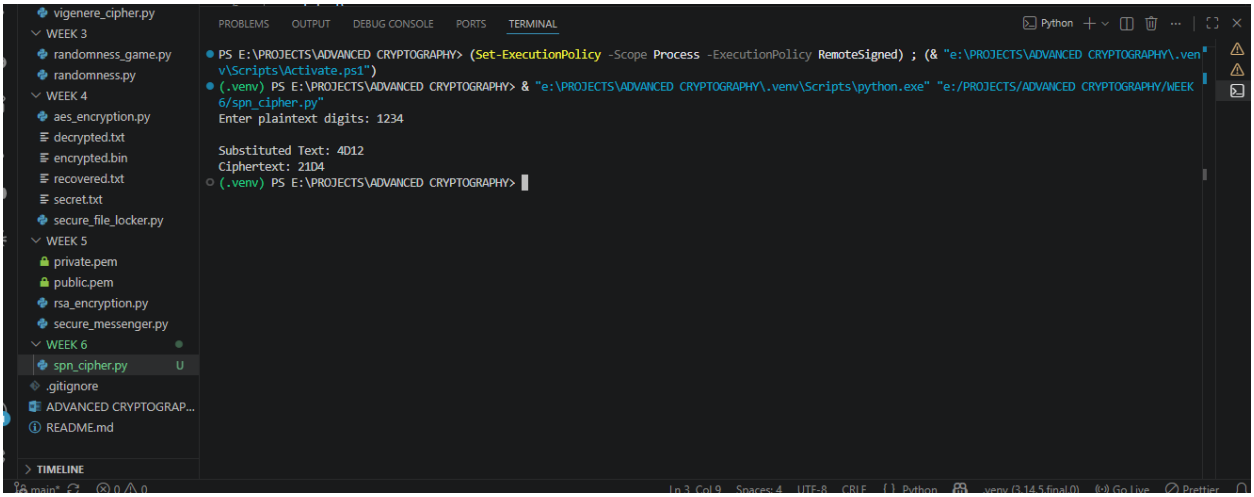


Figure 34: Simple SPN cipher implementation and execution.

CREATIVE TASK 1 - SPN Visualizer

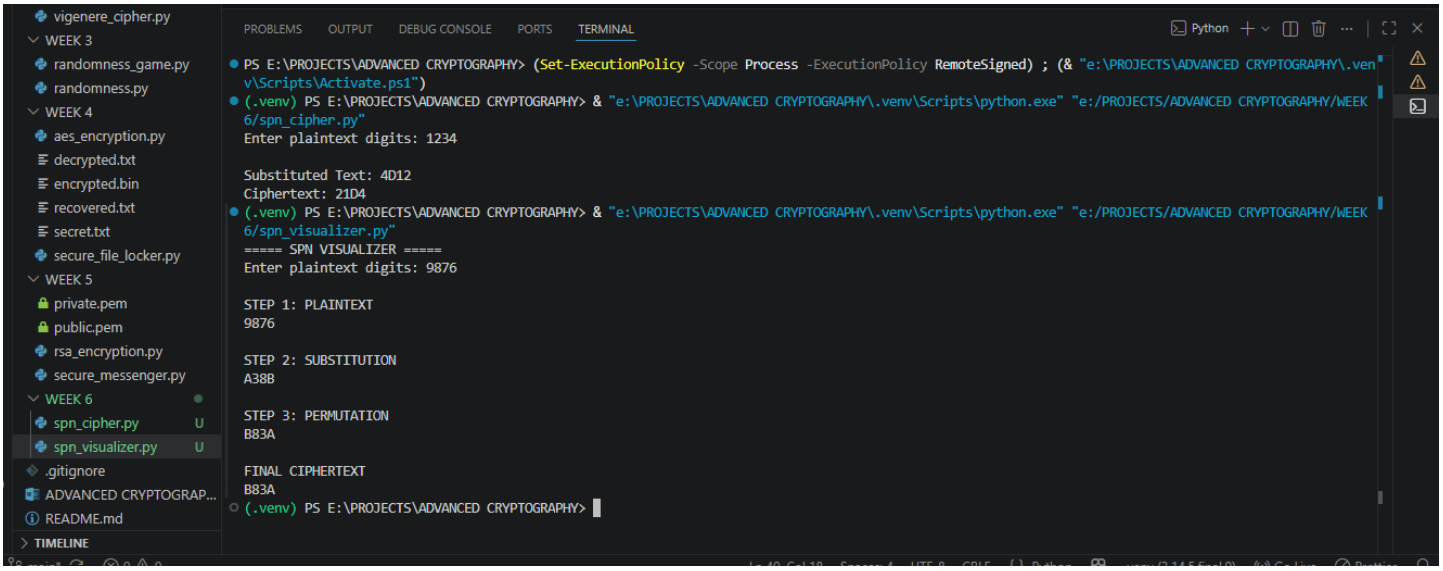


Figure 35: SPN Visualizer illustrating the step-by-step transformation of plaintext through substitution and permutation stages to produce ciphertext.

Creative Task 2 - Avalanche Effect Analyzer

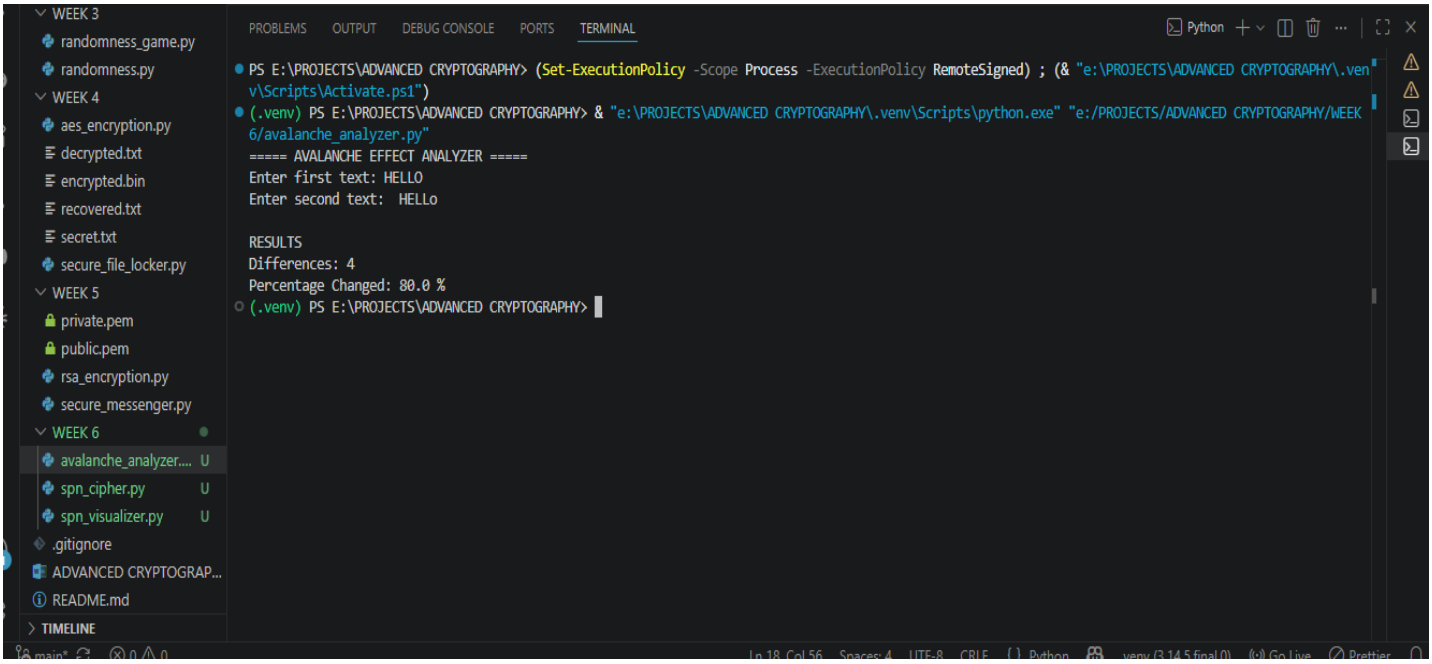


Figure 36: Avalanche Effect Analyzer showing input difference analysis.

REFLECTION

Week 6 explored Substitution-Permutation Networks through manual simulations, S-Box analysis, and SPN implementation. The creative tasks enhanced understanding of cipher structure and demonstrated how small input changes can influence cryptographic outputs.

WEEK 7 - Cryptanalysis Techniques

Class Activity 1: Difference Analysis

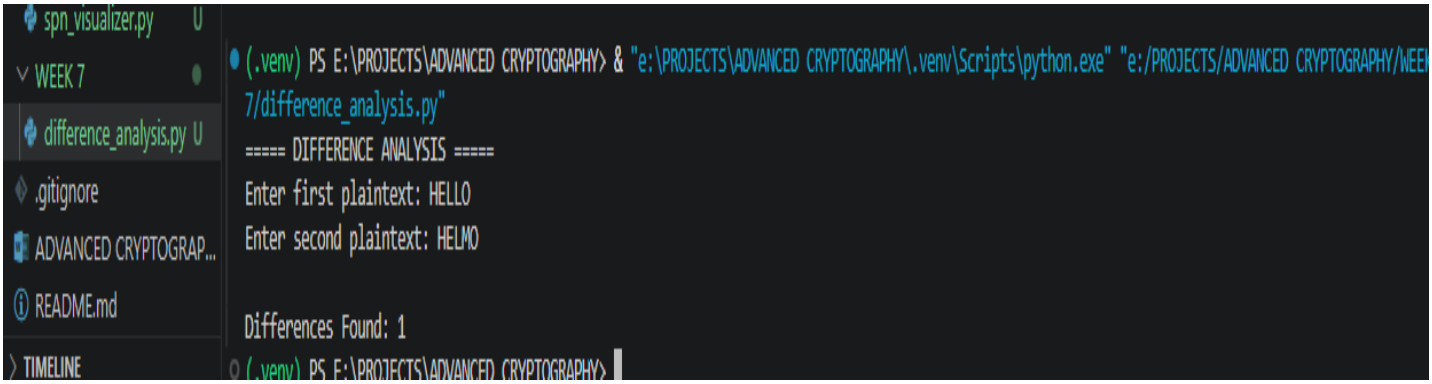


Figure 37:Difference analysis implementation and execution.

Class Activity 2: Frequency Analysis

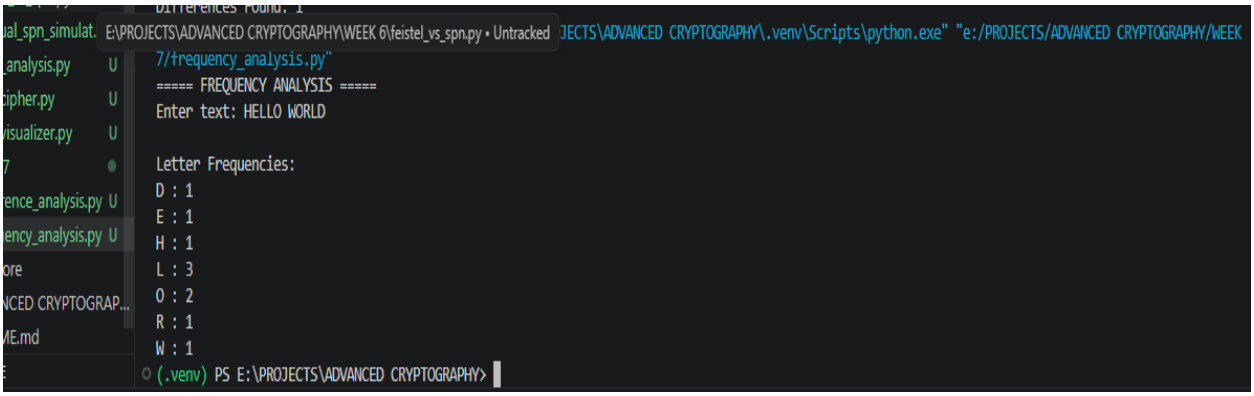


Figure 38 : Frequency analysis showing character occurrence counts.

Class Activity 3: Probability Analysis

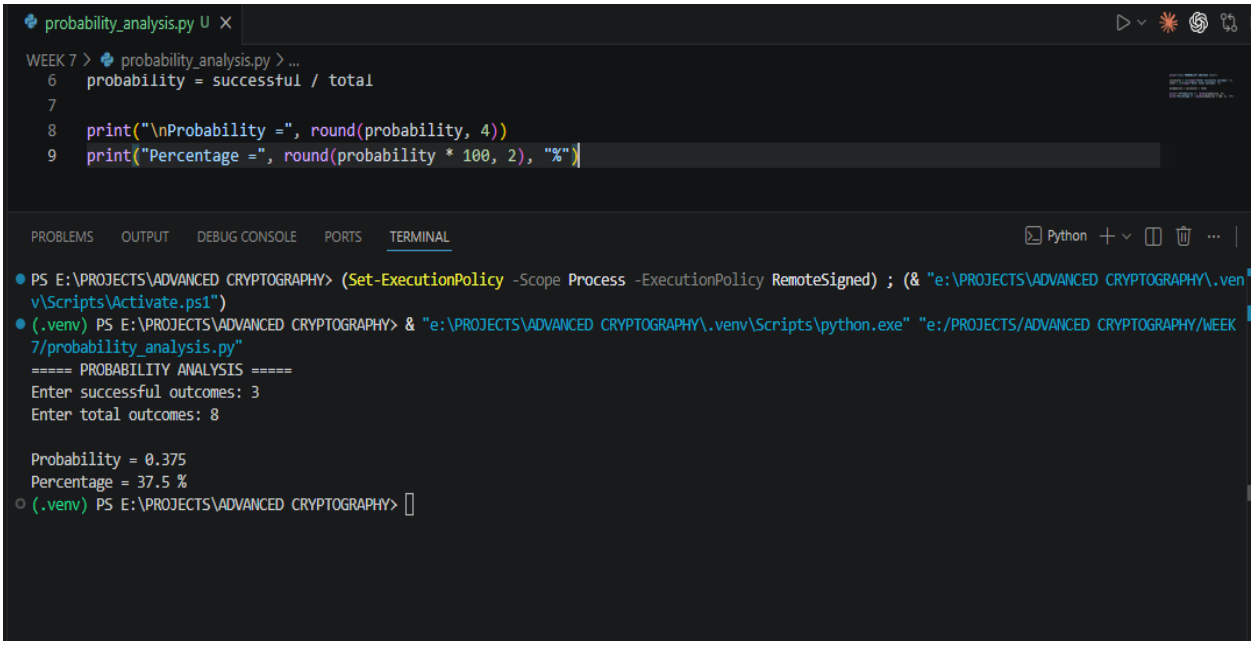


Figure 39: Probability analysis showing likelihood calculation.

Class Activity 4: Differential Cryptanalysis Simulation

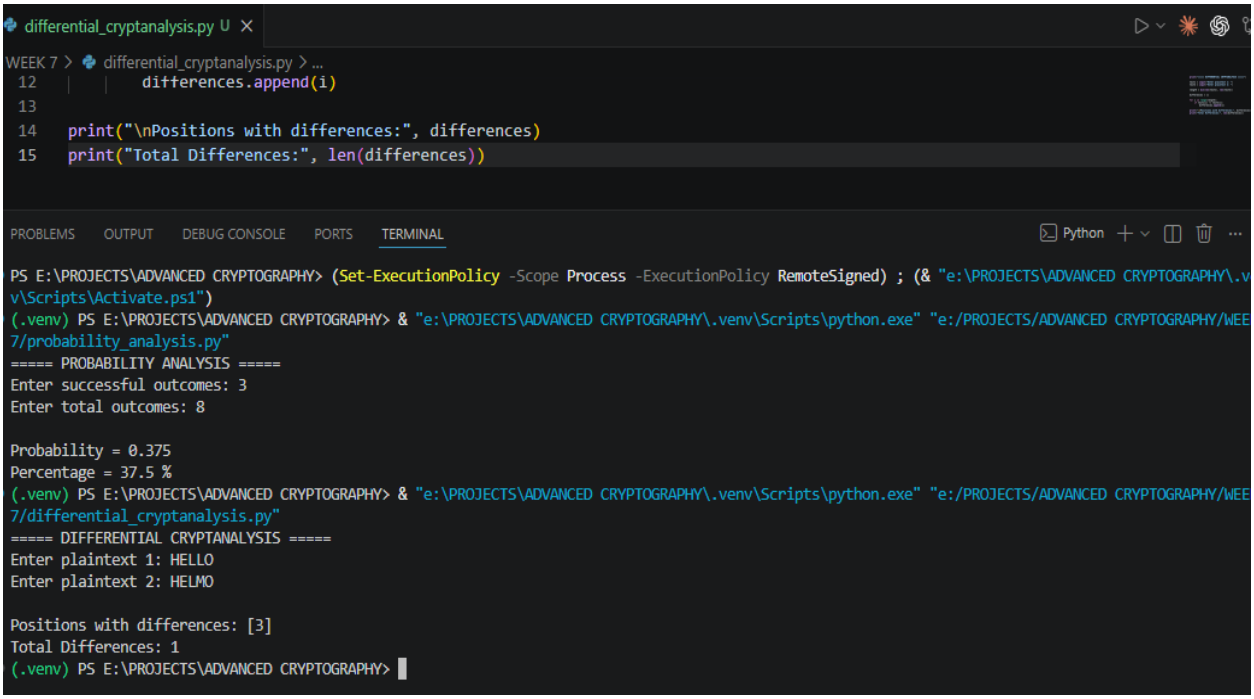


Figure 40:Differential cryptanalysis simulation showing plaintext differences.

CREATIVE TASK – Cryptanalysis Toolkit

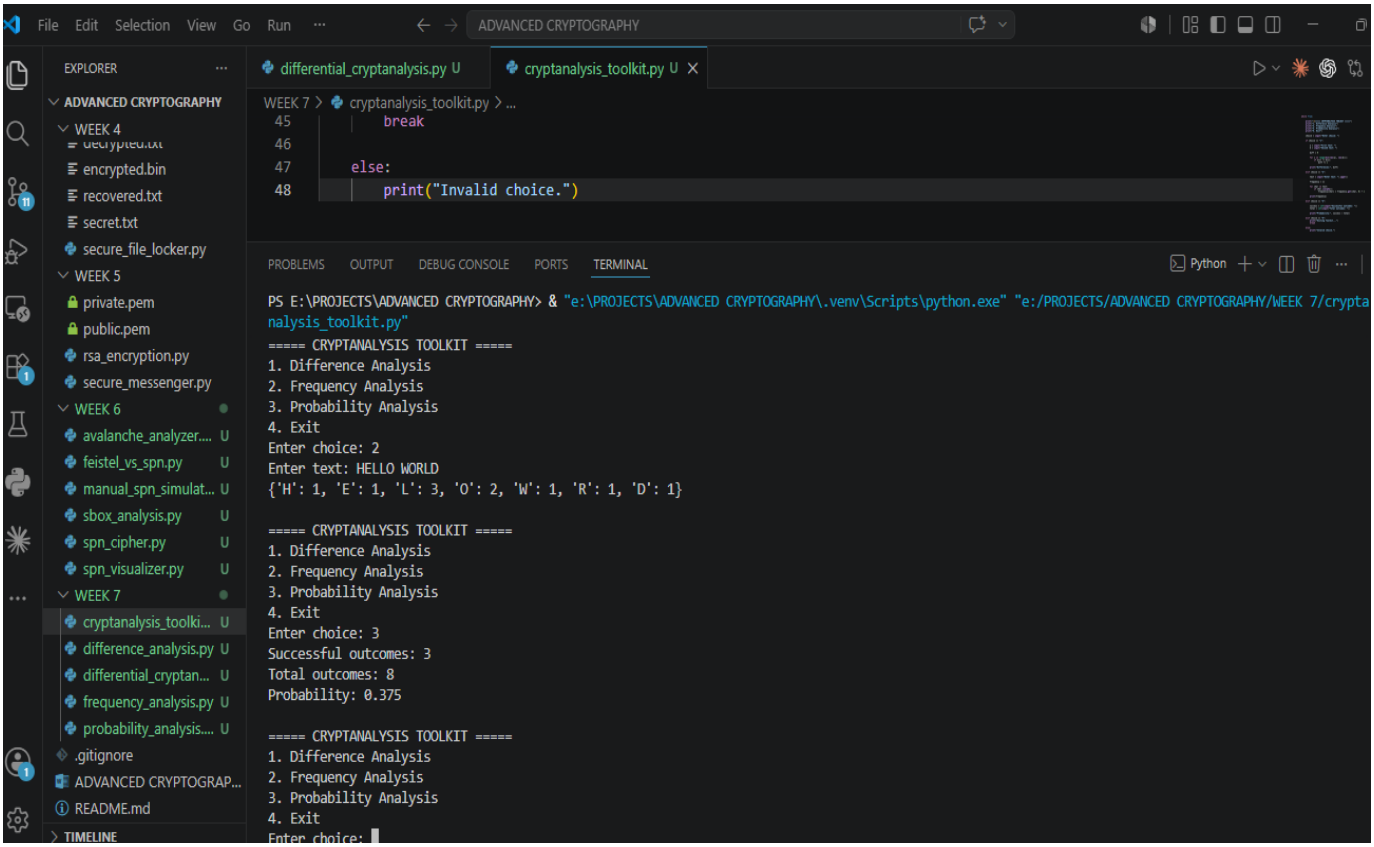


Figure 41: Cryptanalysis Toolkit integrating difference, frequency, and probability analysis techniques.

REFLECTION

Week 7 explored cryptanalysis techniques including difference analysis, frequency analysis, probability calculations, and differential cryptanalysis. The Cryptanalysis Toolkit combined these methods into a single application, improving understanding of how cryptographic systems can be evaluated and attacked.